BlinkAidXGB Documentation

Data labeling

Data labeling is carried out with two helper scripts: "annotationPlots.py" and "annotate_data.py".

The input is a csv file path.

Example – if we want to label a gaze up – gaze down data file we will:

Create a timestamps CSV with the columns "start,stop,label".
Put the path of the csv data file to the "visualize_channels" method (as shown in the main section of the file) and run "annotationPlots.py".
In the pop-up window select the channels to plot, then click "Update plot".
Hover over the event's start, press <space> to copy its timestamp, and paste it into the timestamps CSV. Hover over the end of the event and press the event-specific key (e.g., "U" for gaze-up); paste that value to complete the row.

Important notes:

- Each labeling file begins and ends with "garbage" events (key "G") that capture blinks or irregular behaviour; these rows are dropped during preprocessing.
- You can use any other annotation workflow as long as it outputs a timestamps CSV in the same format.

Annotating raw data

Once the timestamps CSV is ready, put its path in "annotate_data.py" and run it. The function "annotate_data" takes three paths: the raw data file, its timestamps CSV, and the desired output path. It adds a "label" column to every sample (0 = neutral, >0 = event class).

- Important: if you introduce a new class, remember to update the mapping inside "annotate_data.py".

Training

Training is handled by "model_training.py". You select which annotated files (and which subjects) to use via "annotated_data_paths.py" - which contains a dictionary with all the paths of the files we want to train the model on. After training, the script stores the model along with a validation-set performance report.

Training on new data

After recording and labeling new data add the new file paths to "annotated_data_paths.py".

- If it is a brand-new subject, also create a key for that subject in the paths dictionary.
- Include the subject's name in the "subject_list" inside "model_training.py".

Adding a new class

1. Extend the label mapping in "annotate_data.py".
2. Create a new DetectionType entry.
3. Insert the class into the "classes" list in "model_training.py" at the index that matches its numeric label - if the label is 7, it should be the 8th element (list starts from 0).
4. Annotate the data and retrain the model.

Prediction

Real-time predictions are served through the BlinkAid "detect" API. For offline evaluation on CSV files, use "test_model.py".

Model code files description

- annotationPlots.py – graphical interface for creating timestamps file for a specific csv recording (can be done with any other method, such as the BlinkAid annotation interface).
- annotate_data.py – creates labeled CSVs from raw recordings and timestamp files.
- annotated_data_paths.py – central registry of annotated data file paths.
- BlinkAidXGB.py – model class with training, testing, and detection logic.
- model_training.py – main training driver.
- training_helpers.py – shared utility functions.
- continue_training.py – fine-tunes an existing model; to use only when the new data cover all classes.
- test_model.py – evaluates a trained model on labeled CSVs.
- hyperparameter_tuning.py – grid search / optimisation of model hyper-parameters.

Decision Trees, Gradient boosting and XGBoost – a brief explanation

1. Decision trees: the basic building block

A decision tree is a flow-chart that asks a series of "yes / no" questions about features in your data. Each question splits the data into two groups that are more alike internally than before the split. You keep splitting until each final bucket—called a leaf—contains records that can safely share the same prediction.

One tree is easy to interpret but fragile: a shallow tree may miss important patterns, while a deep tree can memorize the training set and behave unpredictably on new data.

2. Gradient boosting: turning many weak trees into one strong model

Gradient Boosting builds a team of very small trees that learn from each other's mistakes:

- Start simple: build a tiny tree that makes a rough guess.
- Measure which cases the tree handled poorly.
- Add a new tiny tree that focuses only on those mistakes.
- Blend the newcomer into the existing team with a small weight, nudging the overall model toward improvement.
- Repeat hundreds of times, each new tree correcting the leftover errors of its predecessors.

The ensemble ends up accurate yet stable because each tree is deliberately limited in depth and influence.

3. XGBoost: "gradient boosting on steroids"

XGBoost (eXtreme Gradient Boosting) follows the same "learn-from-errors" playbook but adds engineering and extra safeguards against over-fitting. XGBoost is considered a SOTA algorithm for tabular datasets.

Some of its benefits are:

- High predictive power on messy data with minimal feature engineering.
- Fast iteration cycles; retraining completes in minutes for rapid experimentation.
- Automatic handling of missing or sparse fields simplifies data pipelines.

4. Key knobs we control (and what they do)

Setting	Typical Range	Intuitive Impact
Learning rate (eta)	0.01–0.3	Smaller = gentler corrections → better generalisation; needs more trees.
Maximum depth / leaves	3–12 levels or 32–256 leaves	Deeper trees uncover finer patterns but raise over-fit risk.

N-estimators	50+	Determines the number of trees in the ensemble.

5. Why XGBoost was the right choice for this project

- Accuracy vs. effort: outperformed simpler baselines without heavy feature engineering.
- Speed to experiment: training time stayed below 10 minutes on our data.
- Production compatibility: loads in milliseconds and provides probability outputs.